

Informe técnico

Asignación de horarios universitarios con GNU Prolog

Alejandro Jiménez Segura

José Mario Castro Cruz

Instituto Tecnológico de Costa Rica

Profesor: Eddy Ramírez Jiménez

10 de octubre de 2026

1. Modelo del problema y representación interna

1.1. Conjuntos y hechos

El dominio del problema (días D , franjas F , profesores P , aulas A y pares curso-grupo C) se representa con hechos dinámicos que `cargar_instancia/1` asierta tras limpiar los de una carga anterior.

Hecho	Significado
<code>dia(D)</code>	Día lectivo, en el orden declarado en <code>CONFIG</code> .
<code>num_franjas(N)</code>	Franjas por día $ F $.
<code>hora_inicio(H), duracion_franja(M)</code>	Hora de inicio y duración en minutos.
<code>aula(A,Cap,Tipo)</code>	Aula con capacidad y tipo.
<code>profesor(P, Nombre, Max, Disp)</code>	<code>Disp</code> es total o una lista de <code>rango(Dia,Desde,Hasta)</code> .
<code>grupo(C, G, Nombre, Insc, Sem,Dur,Req)</code>	Un grupo con sus inscritos, sesiones semanales, duración y tipo requerido.
<code>imparte(C,G,P)</code>	Profesor que imparte el grupo.
<code>aula _ fija(C, G, A), prohibida(C, G, D, F), excluyentes(C1,G1,C2,G2)</code>	Restricciones duras declaradas.
<code>prefiere(C, G, D, F, W), compacta(C,G,W)</code>	Restricciones blandas con su peso.

Los identificadores se normalizan al cargarlos (`normalizar_id/2`): se pasan a minúsculas y el guion se cambia por guion bajo (`IC-1802` se almacena como `ic_1802`), de modo que coinciden con el formato legible por máquina.

1.2. Sesiones

Un grupo con s_c sesiones semanales produce s_c términos

`sesion(Id, Curso, Grupo, Indice, Profesor, Duracion)`

construidos por `sesiones/1`. Cada término es una variable de decisión $X_{c,i}$. El espacio de búsqueda del modelo más ingenuo es

$$\prod_{c \in C} (|A| \cdot |D| \cdot (|F| - d_c + 1))^{s_c},$$

que crece exponencialmente con el número de sesiones.

1.3. Dominio de cada sesión

`dominio/2` calcula, para una sesión, la lista de valores `v(Aula,Dia,FranjaInicio)` que cumplen las restricciones decidibles sin mirar otras sesiones:

- **Capacidad:** capacidad del aula $\geq$ inscritos.
- **Tipo:** laboratorio solo en aulas laboratorio o mixta; teoria solo en teoria o mixta; mixta en cualquier aula.
- **Aula fija:** si el grupo la declara, solo esa aula.
- **Franja de inicio:** entre 1 y $|F| - d_c + 1$.
- **Prohibida:** se descartan los inicios cuya sesión cubriría una franja prohibida.

- **Disponibilidad del profesor:** se exige disponibilidad en *todas* las franjas que cubre la sesión.

El último punto es una decisión de diseño: el enunciado pide que la sesión *comience* en una franja disponible, y el sistema aplica la condición más estricta (véase la sección 11).

1.4. Horario

Un horario es una lista de términos `asignacion(Curso, Grupo, Prof, Aula, Dia, Franja, Dur)`, el mismo formato del archivo legible por máquina. `a_horario/2` lo ordena por curso, grupo, posición cronológica del día y franja.

2. Estrategias de resolución

2.1. Contadores y límite de búsqueda

Los nodos y retrocesos se cuentan con variables globales de GNU Prolog (`g_assign/2`, `g_inc/1`), que no se deshacen al retroceder. Un *nodo* es un intento de asignar un valor a una sesión (en CLP(FD), una decisión de etiquetado); un *retroceso* es un intento cuya continuación falló. Toda la búsqueda con backtracking pasa por un único punto de elección:

```
elegir(V, Dom) :-
    member(V, Dom),
    contar_nodo,
    (   true
    ;   g_inc(retrocesos), fail
    ).
```

`contar_nodo/0` lanza `limite_alcanzado` al superar el límite configurable (por defecto 10^6 nodos). `resolver/3` captura la excepción y devuelve `limite_alcanzado` sin ningún horario parcial.

2.2. Generar y probar (bt_gp)

Estrategia ingenua y referencia de las métricas. Asigna *todas* las sesiones y solo entonces ejecuta `verifica/1`:

```
generar([], []).
generar([S-Dom|Resto], [S-V|Asig]) :-
    elegir(V, Dom),
    generar(Resto, Asig).

buscar(bt_gp, _Opc, Pares, Sol) :-
    generar(Pares, Sol),
    a_horario(Sol, Horario),
    verifica(Horario).
```

2.3. Verificación anticipada (bt)

Cada valor se compara de inmediato contra las sesiones ya asignadas:

```
buscar_anticipada([], _, _, []).
buscar_anticipada([S-Dom|Resto], Sim, Prev, [S-V|Asig]) :-
```

```
elegir(V, Dom),
\+ ( member(S2-V2, Prev), inconsistente(Sim, S, V, S2, V2) ),
buscar_anticipada(Resto, Sim, [S-V|Prev], Asig).
```

`inconsistente/5` incluye los conflictos duros de aula, profesor, grupo y `excluyentes` y, si se activa, las rupturas de simetría.

2.4. Forward checking con MRV y LCV

Con `mrp`, tras cada asignación se podan los dominios de las sesiones pendientes (`filtrar/5`); si algún dominio queda vacío se retrocede de inmediato, y la siguiente variable es la de menor dominio. Con `lcv` se usa forward checking con orden estático de variables y los valores de cada dominio se ordenan de menor a mayor número de valores descartados en las sesiones pendientes (`ordenar_valores/6`).

2.5. Verificación independiente

`verifica/1` no reutiliza las rutinas de la búsqueda: expande cada asignación a celdas `c(Tipo, Recurso, Dia, Franja)` para aula, profesor y grupo y detecta celdas repetidas. Comprueba además número de sesiones por grupo, rangos, capacidad, tipo, disponibilidad, máximo de franjas, `aula_fija`, `prohibida` y `excluyentes`. `violaciones/2` devuelve la lista de lo que falla.

2.6. CLP(FD)

Modelo. Por cada sesión hay *una* variable X con dominio $1..N$, donde N es el tamaño del dominio de `dominio/2`. Con `fd_element/3` se obtienen dos variables derivadas:

$$A = \text{índice del aula}, \quad T = \text{día} \cdot K + \text{franja}, \quad K = |F| + 1.$$

T es un *tiempo absoluto*: como cada sesión cabe en su día, dos intervalos $[T, T + d)$ de días distintos nunca se solapan, y el no solapamiento se reduce a una disyunción lineal. Aulas y días se indexan con enteros consecutivos desde 0.

```
construir_vars([S-Dom|Ps], [fv(S, X, A, T, Dom, As)|Vs]) :-
    length(Dom, N), k_dia(K),
    tablas(Dom, K, LA, LT, As0), sort(As0, As),
    fd_domain(X, 1, N),
    fd_element(X, LA, A),
    fd_element(X, LT, T),
    construir_vars(Ps, Vs).
```

Traducción de las restricciones.

Restricción	Traducción
Capacidad, tipo, aula_fija, prohibida, disponibilidad	Filtrado del dominio de X (extensional).
Mismo profesor, mismo grupo, excluyentes	$T_1 + d_1 \leq T_2 \vee T_2 + d_2 \leq T_1$.
Pares con aulas en común (distinto profesor y grupo)	$(A_1 = A_2) \Rightarrow (T_1 + d_1 \leq T_2 \vee T_2 + d_2 \leq T_1)$ (reificación con $\#=>$).
Redundantes	<code>fd_all_different/1</code> sobre los T de cada profesor y de cada grupo, y sobre $A \cdot B + T$ en todo el sistema ($B = D \cdot K + 1$).
Simetría	$T_1 < T_2$ entre sesiones intercambiables (sección 5).

Restricciones no puramente extensionales. Las restricciones unarias (capacidad, tipo, aula fija, prohibida, disponibilidad) se resuelven reduciendo el dominio y con las tablas de `fd_element/3`. En cambio, el no solapamiento de aula relaciona dos sesiones y depende del valor de las aulas de ambas; no se puede expresar con una tabla sobre una sola variable. Se maneja con una implicación reificada y una disyunción lineal sobre T , que el solucionador propaga más débilmente que una tabla; para reforzarla se añaden las restricciones redundantes con `fd_all_different/1`. Las restricciones de pares se postean con recursión explícita y no con `forall/2` ni `findall/3`, porque estos deshacen los efectos sobre las variables FD.

2.7. Etiquetado: propio frente a `fd_labeling/2`

- **Etiquetado propio** (por defecto). Cada decisión es binaria: $X = W$ o $X \neq W$. Cuenta nodos y retrocesos, respeta el límite y es el único que usa el branch and bound.
- `fd_labeling/2` de GNU Prolog, con la opción `backtracks(B)` que entrega los retrocesos. No cuenta nodos ni respeta el límite, porque no tiene opción de límite.

3. Restricciones duras y blandas

3.1. Restricciones duras

Restricción	Cómo se garantiza
No solapamiento de grupo, profesor y aula	<code>incompatibles/4</code> en la búsqueda; en CLP(FD), disyunciones y <code>fd_all_different/1</code> (sección 2.6).
Capacidad y tipo de aula	Dominio de cada sesión (sección 1.3).
Disponibilidad del profesor	Dominio, sobre todas las franjas de la sesión.
Máximo de franjas del profesor	La carga es constante (suma de las duraciones de sus sesiones): se comprueba antes de buscar en <code>diagnostico_previo/2</code> y al final en <code>verifica/1</code> .
aula_fija	Se reduce el dominio a esa aula.
prohibida	Se descartan los inicios que cubren la franja prohibida.
excluyentes	Conflicto entre las dos sesiones (<code>excluyentes_par/4</code>), igual que un mismo grupo.

3.2. Restricciones blandas y costo

`prefiere` penaliza con w si el grupo no tiene ninguna sesión que comience en la franja indicada; `compacta` penaliza con w por cada hueco del grupo, entendido como una franja libre estrictamente entre la primera y la última ocupada por el grupo en un mismo día. El costo que minimiza el sistema

es

$$\text{costo}(S) = \sum_{\text{prefiere}} w \cdot [\text{no cumplida}] + \sum_{\text{compacta}} w \cdot \text{huecos}(g).$$

Los huecos de profesor y de grupo implícitos (opcionales) se calculan y se reportan en el resumen, pero no forman parte del costo optimizado.

4. Heurísticas y mecanismos de poda

Heurística	Implementación explícita	Opción de <code>fd_labeling/2</code>
<code>original</code>	Sesiones en orden de lectura.	<code>variable_method(standard)</code> .
<code>mrp, ff</code>	Primero la sesión con menor dominio (forward checking en <code>bt; fd_size/2</code> en <code>CLP(FD)</code>).	<code>variable_method(first_fail)</code> .
<code>ffc</code>	En <code>CLP(FD)</code> , reordenar por grado y luego aplicar <code>ff</code> .	<code>variable_method(most_constrained)</code> .
<code>grado</code>	Primero la sesión con más conflictos con otras (mismo grupo, mismo profesor, excluyentes), calculado una vez.	Orden previo y etiquetado estándar (aproximación).
<code>demanda</code>	Primero los grupos con más inscritos y, a igualdad, mayor duración.	Orden previo y etiquetado estándar (aproximación).
<code>lcv</code>	El valor que descarta menos valores en las demás sesiones.	No disponible en GNU Prolog; con <code>labeling(fd)</code> se reporta como error de argumentos.

Por qué `ff` equivale a MRV. MRV elige la variable con menor número de valores aún posibles. Como el modelo tiene *una* variable por sesión y su dominio es el conjunto de ternas `v(Aula,Dia,Franja)` aún posibles, el tamaño del dominio de X es el número de valores restantes; por eso `first_fail` coincide con MRV. Con tres variables por sesión (aula, día, franja) la equivalencia no se mantendría.

Mecanismos de poda.

- **Verificación anticipada:** descarta una asignación parcial al chocar con una ya hecha.
- **Forward checking:** descarta valores de las sesiones pendientes y retrocede cuando un dominio queda vacío.
- **Propagación en `CLP(FD)`:** disyunciones, implicaciones reificadas y `fd_all_different/1` reducen los dominios con cada decisión.
- **Cota inferior del branch and bound** (sección 6).

5. Reducción de simetría

Se implementaron dos mecanismos, activables con `simetria(no|intra|inter|si)`.

Ordenamiento intra-grupo. Las sesiones de un grupo son intercambiables, así que la de índice menor debe ir antes en el tiempo. Cada franja se convierte en un tiempo cronológico $t = \text{posición del día} \cdot 1000 + \text{franja}$, y `rompe_intra/4` rechaza el par si el orden se invierte. En `CLP(FD)` se postea $T_1 < T_2$.

Ordenamiento inter-grupo. Dos grupos son *equivalentes* (`equiv_g/4`) si tienen el mismo profesor, inscritos, sesiones semanales, duración y tipo, y *ninguno* tiene restricciones declaradas. Intercambiar sus horarios produce otra solución válida del mismo costo, así que se fija el orden de sus primeras sesiones (`rompe_inter/4`). La exigencia de que no haya restricciones declaradas garantiza que la ruptura no elimina todas las soluciones. Ninguno de los dos mecanismos es una restricción del problema, por lo que `verifica/1` no los comprueba.

6. Branch and bound

La optimización de las restricciones blandas se implementó sobre CLP(FD) (`--optimizar`). Se recorre el árbol del etiquetado propio y en cada nodo se calcula una cota inferior del costo; se poda si la cota es mayor o igual que el mejor costo conocido. En cada hoja se calcula el costo exacto con `costo_blandas/2`; si mejora, se guarda y se sigue buscando hasta agotar el árbol o hallar costo 0. La cota es siempre válida:

- **prefiere:** suma w cuando ninguna sesión del grupo puede ya comenzar en esa franja.
- **compacta:** suma $w \cdot \text{huecos}$ solo cuando todas las sesiones del grupo ya están asignadas, porque antes los huecos aún pueden llenarse.

El estado de optimalidad es **si** (árbol agotado o costo 0: solución **óptima**), **no** (límite alcanzado con una solución en mano: mejor conocida, sin garantía) o **n_a** (sin optimización). Si se alcanza el límite sin ninguna solución, se informa `LIMITE ALCANZADO`.

7. Lectura, validación y escritura de archivos

7.1. Lectura

La lectura se construyó con predicados básicos, sin utilidades externas:

- `leer_lineas/2` lee carácter a carácter con `get_char/2` hasta el fin de línea o de archivo y descarta `\r` (archivos CRLF).
- `split_on/3` divide por un separador y conserva los campos vacíos finales.
- `recortar/2` elimina espacios y tabuladores de cada campo.
- `clasificar_linea/2` clasifica la línea como `blanco`, `seccion(Nombre)`, `comentario` o `registro(Campos)`.

`procesar_registro/3` tiene una cláusula por sección y tipo de restricción; si un registro no se puede procesar lanza `error_registro(Linea, Seccion, Campos)`.

7.2. Validación con número de línea

`valida/1` se ejecuta *antes* de cargar la instancia. Recolecta todos los errores, los ordena y los reporta con la forma `linea(N, Razon)` (0 indica un error global). Los pases son: estructura (registros sin sección, secciones desconocidas), secciones duplicadas, `CONFIG` (claves obligatorias y únicas, valores válidos), universo de días, aulas, profesores y grupos, registros de datos (aridad, números, tipos, rangos y referencias), duplicados y cobertura de `IMPARTE`. Los enteros se validan con `safe_int/2`, que acepta solo dígitos decimales.

Error de formato	Término que lo reporta
Registro antes de cualquier sección	registro_sin_seccion
Falta una clave obligatoria en CONFIG	clave_config_faltante(K)
Campo numérico no entero o negativo	franjas _ invalida, duracion _ invalida, aula_invalida, curso_invalido, peso_invalido
Código de día no declarado	profesor_invalido, dia_franja_invalida
Franja fuera de $1 \dots F $	profesor_invalido, dia_franja_invalida
Tipo de aula o de requerimiento desconocido	aula_invalida, curso_invalido
Referencia inexistente	grupo_inexistente, profesor_inexistente, aula_inexistente
Par (código, grupo) duplicado	par_duplicado
Sesión que excede el último índice de franja	duracion_excede_franjas, sesion_excede_dia

Ejemplo de reporte (por `stderr`):

```

Instancia invalida: 2 error(es)
  linea 18: grupo_inexistente(x_1-c1)
  linea 19: grupo_inexistente(y_1-c1)

```

7.3. Escritura de la salida

`main/3` produce el archivo de cinco bloques: encabezado (estado `SOLUCION ENCONTRADA`, `SIN SOLUCION`, `LIMITE ALCANZADO` o `INSTANCIA INVALIDA`), detalle de asignaciones, horario por grupo y por profesor (y por aula con `por_aula`), resumen con métricas, y diagnóstico cuando no hay solución. Con `salida_prolog(Ruta)` escribe además los hechos `asignacion/7` a partir de la misma lista ordenada, por lo que ambos archivos son idénticos en contenido. Con el límite alcanzado no se produce ningún horario parcial: el archivo contiene solo el encabezado con `LIMITE ALCANZADO` y un mensaje.

8. Manejo de instancias inválidas o insatisfacibles

Situación	Comportamiento	Código
Archivo inexistente o ilegible	<code>error_instancia(no_legible)</code> por <code>stderr</code> .	2
Errores de sintaxis o coherencia	Lista completa con número de línea; salida con <code>INSTANCIA INVALIDA</code> .	2
Inconsistente por construcción	<code>diagnostico _ previo / 2</code> , sin buscar; bloque <code>DIAGNOSTICO</code> .	1
Insatisfacible aunque consistente	La búsqueda agota el espacio; bloque <code>DIAGNOSTICO</code> con la restricción culpable.	1
Límite de búsqueda	<code>LIMITE ALCANZADO</code> , sin horario parcial.	3
Salida no escribible	<code>error_escritura</code> .	4

Diagnóstico previo. `diagnostico_previo/2` detecta, antes de buscar, de la causa más barata a la más costosa: `grupo_excede_franjas` ($s_c > |D| \cdot \lfloor |F|/d_c \rfloor$), `carga_profesor`, `disponibilidad_insuficiente` y `dominio_vacio` (sesión sin ningún valor posible por capacidad, tipo, aula fija, etc.).

Bloque de diagnóstico y restricción culpable. Cuando no hay solución, el archivo contiene la razón inmediata, el número de sesiones frente a las franjas disponibles, las franjas-aula requeridas

frente a las disponibles, el grupo con mayor demanda y el profesor con mayor carga. `restricciones_culpables/1` quita *sola* cada restricción dura declarada (`aula_fija`, `prohibida`, `excluyentes`), vuelve a resolver con `bt` y MRV (límite de 200 000 nodos) y, si entonces hay solución, la informa como culpable. Al terminar restaura los hechos en su orden original. Si ninguna resuelve la instancia, el diagnóstico lo indica y señala que el límite está en aulas, franjas, disponibilidad o carga del profesor.

9. Resultados experimentales

9.1. Metodología

- **Equipo:**
 - Sistema operativo: Linux Mint 22.3 – Cinnamon, 64 bits.
 - Procesador: AMD Ryzen 7 7800X3D de 8 núcleos ($\times 8$).
 - Tarjeta gráfica: NVIDIA Corporation TU117 GeForce GTX 1650.
 - Memoria: 15.4 GB.
- **Versión de GNU Prolog:** 1.5.0.
- **Instancias:** 10, 20, 40 y 80 sesiones para el escalamiento, más `blandas` para el branch and bound.
- **Límite de búsqueda:** 200 000 nodos en las comparaciones.
- **Repeticiones:** 5. Los tiempos de 1 o 2 ms están dentro de la resolución del reloj y no son interpretables.

9.2. Métricas

$$R_e = \frac{N_e}{T_e} \quad \text{tasa de exploración (nodos por segundo),} \quad (1)$$

$$S = \frac{T_{\text{ingenue}}}{T_{\text{optimizado}}} \quad \text{aceleración,} \quad (2)$$

$$E = \frac{N_{\text{ingenue}} - N_{\text{optimizado}}}{N_{\text{ingenue}}} \quad \text{eficiencia de la poda.} \quad (3)$$

T_e es el tiempo de CPU medido con `statistics/2` y N_e el número de nodos del contador explícito. La referencia ingenua es *generar y probar*. Cuando la referencia alcanza el límite, S y E son cotas inferiores (prefijo $\geq$); las filas que alcanzan el límite muestran un guion. En CLP(FD) con etiquetado propio un nodo es una decisión de etiquetado; en backtracking es el intento de asignar una sesión, por lo que son comparables solo en orden de magnitud. Con `fd_labeling/2` el contador de nodos es 0 y E no se calcula.

9.3. Configuraciones comparadas

N.º	Configuración	Cómo se ejecuta
1	Interpretada con <code>gprolog</code>	<code>./horarios.sh ...</code>
2	Compilada con <code>gplc</code>	<code>./horarios_nativo ...</code>
3	Backtracking sin heurísticas	<code>bt_gp</code> ; <code>bt</code> con orden original
4	Verificación anticipada con MRV	<code>bt --heuristica=mrv</code>
5	Dominios finitos sin blandas	<code>clpfd --heuristica=ff</code>
6	Dominios finitos con branch and bound	<code>clpfd --optimizar</code>

9.4. Comparación de todas las configuraciones

Configuración	Resultado	Nodos	Retroc.	CPU (ms)	S	E	Verifica	Costo
generar y probar	limite_alcanzado	200001	199991	12856	-	-	-	-
anticipada, original	solucion	55	45	0	12856.0	1.0	valida	0
anticipada, original+intra	solucion	55	45	0	12856.0	1.0	valida	0
anticipada, original+inter	solucion	55	45	1	12856.0	1.0	valida	0
anticipada, original+sim	solucion	55	45	0	12856.0	1.0	valida	0
anticipada, grado+sim	solucion	49	39	1	12856.0	1.0	valida	0
anticipada, demanda+sim	solucion	43	33	0	12856.0	1.0	valida	3
FC + MRV	solucion	10	0	1	12856.0	1.0	valida	3
FC + MRV + sim	solucion	10	0	1	12856.0	1.0	valida	3
FC + LCV	solucion	10	0	27	476.14999999999998	1.0	valida	3
CLP(FD), original	solucion	36	26	1	12856.0	1.0	valida	0
CLP(FD), ff	solucion	52	42	0	12856.0	1.0	valida	3
CLP(FD), ff+sim	solucion	28	18	1	12856.0	1.0	valida	3
CLP(FD), ffc+sim	solucion	28	18	1	12856.0	1.0	valida	3
CLP(FD), lcv+sim	solucion	10	0	1	12856.0	1.0	valida	3
CLP(FD), ff + B&B	solucion	81	71	1	12856.0	1.0	valida	0
CLP(FD) fd_labeling, original	solucion	0	26	0	12856.0	1.0	valida	0
CLP(FD) fd_labeling, ff	solucion	0	42	0	12856.0	1.0	valida	3
CLP(FD) fd_labeling, ffc	solucion	0	42	1	12856.0	1.0	valida	3

Cuadro 1: Instancia **campus-central** (10 sesiones).

Configuración	Resultado	Nodos	Retroc.	CPU (ms)	S	E	Verifica	Costo
generar y probar	limite_alcanzado	200001	199981	27241	-	-	-	-
anticipada, original	solucion	225	205	1	27241.0	0.999	valida	2
anticipada, original+intra	solucion	225	205	0	27241.0	0.999	valida	2
anticipada, original+inter	solucion	236	216	0	27241.0	0.999	valida	2
anticipada, original+sim	solucion	236	216	1	27241.0	0.999	valida	2
anticipada, grado+sim	solucion	1362	1342	2	13620.5	0.9929999999999999	valida	1
anticipada, demanda+sim	solucion	2216	2196	3	9080.329999999999	0.9889999999999999	valida	2
FC + MRV	solucion	20	0	5	5448.1999999999998	1.0	valida	2
FC + MRV + sim	solucion	37	17	12	2270.0799999999999	1.0	valida	2
FC + LCV	solucion	20	0	305	89.310000000000002	1.0	valida	2
CLP(FD), original	solucion	172	152	2	13620.5	0.999	valida	2
CLP(FD), ff	solucion	179	159	2	13620.5	0.999	valida	2
CLP(FD), ff+sim	solucion	92	72	1	27241.0	1.0	valida	2
CLP(FD), ffc+sim	solucion	92	72	1	27241.0	1.0	valida	2
CLP(FD), lcv+sim	solucion	20	0	8	3405.1199999999999	1.0	valida	1
CLP(FD), ff + B&B	solucion	402	382	12	2270.0799999999999	0.998	valida	0
CLP(FD) fd_labeling, original	solucion	0	152	1	27241.0	1.0	valida	2
CLP(FD) fd_labeling, ff	solucion	0	159	2	13620.5	1.0	valida	2
CLP(FD) fd_labeling, ffc	solucion	0	159	1	27241.0	1.0	valida	2

Cuadro 2: Instancia **medio20** (20 sesiones).

Configuración	Resultado	Nodos	Retroc.	CPU (ms)	<i>S</i>	<i>E</i>	Verifica	Costo
generar y probar	limite_alcanzado	200001	199961	48429	-	-	-	-
anticipada, original	solucion	782	742	2	24214.5	0.996	valida	11
anticipada, original+intra	solucion	2987	2947	6	8071.5	0.9849999999999999	valida	11
anticipada, original+inter	solucion	782	742	4	12107.25	0.996	valida	11
anticipada, original+sim	solucion	2987	2947	6	8071.5	0.9849999999999999	valida	11
anticipada, grado+sim	solucion	673	633	2	24214.5	0.997	valida	14
anticipada, demanda+sim	solucion	1049	1009	4	12107.25	0.995	valida	14
FC + MRV	solucion	40	0	27	1793.6700000000001	1.0	valida	14
FC + MRV + sim	solucion	42	2	45	1076.2	1.0	valida	14
FC + LCV	solucion	40	0	1637	29.579999999999998	1.0	valida	14
CLP(FD), original	solucion	681	641	8	6053.6199999999999	0.997	valida	11
CLP(FD), ff	solucion	743	703	8	6053.6199999999999	0.996	valida	14
CLP(FD), ff+sim	solucion	518	478	7	6918.4300000000003	0.997	valida	14
CLP(FD), ffc+sim	solucion	516	476	7	6918.4300000000003	0.997	valida	14
CLP(FD), lcv+sim	solucion	40	0	31	1562.23	1.0	valida	11
CLP(FD), ff + B&B	solucion	3204	3204	171	283.20999999999998	0.9839999999999999	valida	4
CLP(FD) fd_labeling, original	solucion	0	641	5	9685.7999999999993	1.0	valida	11
CLP(FD) fd_labeling, ff	solucion	0	703	4	12107.25	1.0	valida	14
CLP(FD) fd_labeling, ffc	solucion	0	703	5	9685.7999999999993	1.0	valida	14

Cuadro 3: Instancia **medio40** (40 sesiones).

Configuración	Resultado	Nodos	Retroc.	CPU (ms)	<i>S</i>	<i>E</i>	Verifica	Costo
generar y probar	limite_alcanzado	200001	199921	118789	-	-	-	-
anticipada, original	solucion	3346	3266	20	5939.4499999999998	0.9829999999999998	valida	21
anticipada, original+intra	solucion	8946	8866	27	4399.5900000000001	0.9549999999999996	valida	21
anticipada, original+inter	solucion	3346	3266	33	3599.6700000000001	0.9829999999999998	valida	21
anticipada, original+sim	solucion	8946	8866	41	2897.29	0.9549999999999996	valida	21
anticipada, grado+sim	solucion	13244	13164	39	3045.8699999999999	0.9340000000000005	valida	23
anticipada, demanda+sim	solucion	4123	4043	36	3299.6900000000001	0.9789999999999998	valida	23
FC + MRV	solucion	80	0	156	761.47000000000003	1.0	valida	18
FC + MRV+ sim	solucion	111	31	292	406.81	0.999	valida	18
FC + LCV	solucion	80	0	17162	6.9199999999999999	1.0	valida	23
CLP(FD), original	solucion	3040	2960	45	2639.7600000000002	0.9849999999999999	valida	21
CLP(FD), ff	solucion	3209	3129	49	2424.27	0.9839999999999999	valida	27
CLP(FD), ff+sim	solucion	2155	2075	38	3126.0300000000002	0.9889999999999999	valida	21
CLP(FD), ffc+sim	solucion	2461	2381	43	2762.5300000000002	0.9879999999999999	valida	18
CLP(FD), lcv+sim	solucion	80	0	164	724.32000000000005	1.0	valida	25
CLP(FD), ff + B&B	solucion	36982	36902	6475	18.350000000000001	0.8149999999999995	valida	0
CLP(FD) fd_labeling, original	solucion	0	2960	21	5656.6199999999999	1.0	valida	21
CLP(FD) fd_labeling, ff	solucion	0	3201	22	5399.5	1.0	valida	23
CLP(FD) fd_labeling, ffc	solucion	0	3120	23	5164.7399999999998	1.0	valida	21

Cuadro 4: Instancia **grande80** (80 sesiones).

Configuración	Resultado	Nodos	Retroc.	CPU (ms)	<i>S</i>	<i>E</i>	Verifica	Costo
generar y probar	limite_alcanzado	200001	199991	13135	-	-	-	-
anticipada, original	solucion	54	44	0	13135.0	1.0	valida	22
anticipada, original+intra	solucion	54	44	1	13135.0	1.0	valida	22
anticipada, original+inter	solucion	54	44	1	13135.0	1.0	valida	22
anticipada, original+sim	solucion	54	44	0	13135.0	1.0	valida	22
anticipada, grado+sim	solucion	53	43	0	13135.0	1.0	valida	17
anticipada, demanda+sim	solucion	55	45	0	13135.0	1.0	valida	17
FC + MRV	solucion	10	0	1	13135.0	1.0	valida	16
FC + MRV + sim	solucion	10	0	0	13135.0	1.0	valida	16
FC + LCV	solucion	10	0	8	1641.8800000000001	1.0	valida	16
CLP(FD), original	solucion	29	19	0	13135.0	1.0	valida	22
CLP(FD), ff	solucion	29	19	0	13135.0	1.0	valida	16
CLP(FD), ff+sim	solucion	23	13	0	13135.0	1.0	valida	16
CLP(FD), ffc+sim	solucion	23	13	1	13135.0	1.0	valida	16
CLP(FD), lcv+sim	solucion	10	0	1	13135.0	1.0	valida	17
CLP(FD), ff + B&B	solucion	360	360	22	597.04999999999995	0.998	valida	4
CLP(FD) fd_labeling, original	solucion	0	19	1	13135.0	1.0	valida	22
CLP(FD) fd_labeling, ff	solucion	0	19	1	13135.0	1.0	valida	16
CLP(FD) fd_labeling, ffc	solucion	0	19	0	13135.0	1.0	valida	16

Cuadro 5: Instancia **blandas** (la columna Costo muestra el efecto del branch and bound).

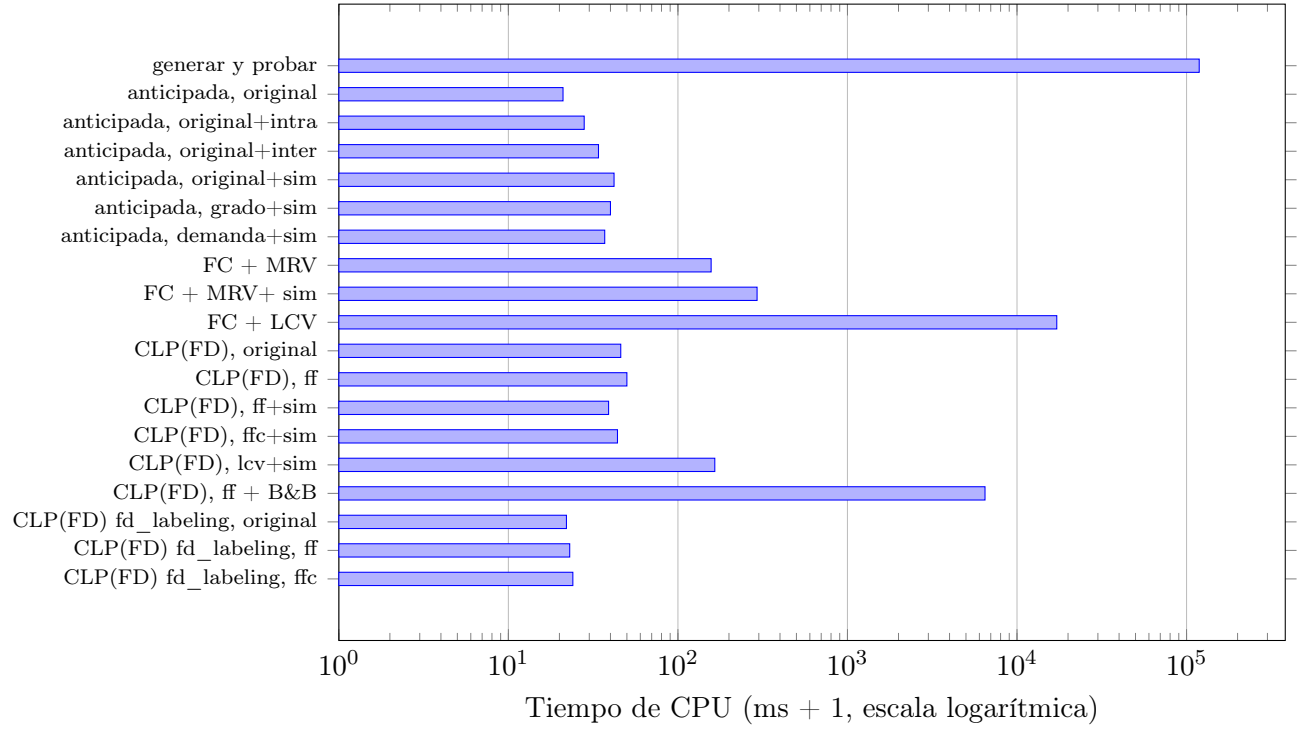


Figura 1: Tiempo de CPU por configuración en **grande80**.

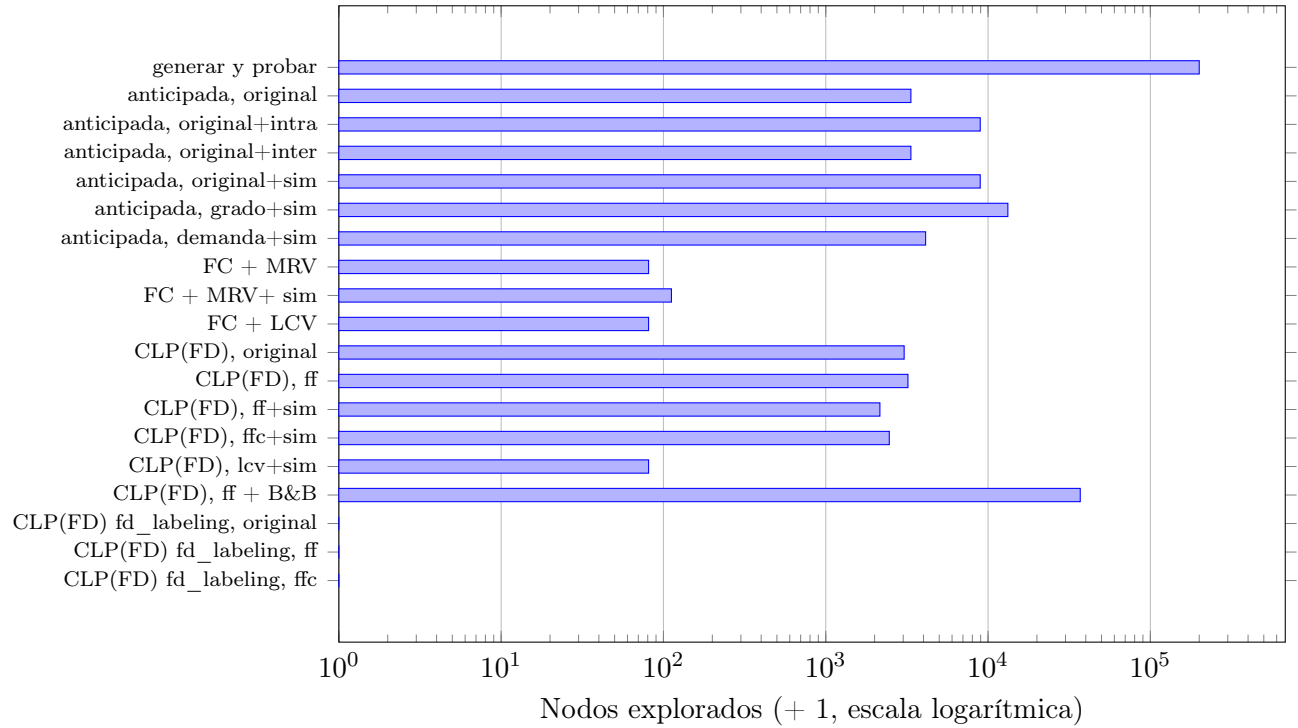


Figura 2: Nodos explorados por configuración en **grande80**.

9.5. Escalamiento con el tamaño

Nodos explorados según el número de sesiones.

Configuración	10	20	40	80
Generar y probar	200001	200001	200001	200001
bt + MRV (FC)	10	20	40	80
CLP(FD) ff	52	179	743	3209
CLP(FD) ff + branch and bound	81	402	3204	36982

Tiempo de CPU (ms) según el número de sesiones.

Configuración	10	20	40	80
Generar y probar	12856	27241	48429	118789
bt + MRV (FC)	1	5	27	156
CLP(FD) ff	0	2	8	49
CLP(FD) ff + branch and bound	1	12	171	6475

9.6. Tiempo de CPU frente a tiempo de pared

Contraste entre el tiempo de CPU de `statistics/2` y el de pared de la orden `time` (valor `real`).

Instancia	Configuración	CPU (resumen)	Pared (time)	Diferencia
medio40	bt --heuristica=mrv	27	0.14 s	0.113 s
grande80	bt --heuristica=mrv	156	0.28 s	0.124 s
grande80	clpfd --heuristica=ff	49	0.28 s	0.231 s

La diferencia es el costo fijo de arrancar `gprolog`, compilar a bytecode los archivos fuente y leer y validar la instancia, que no aparecen en el tiempo de CPU de la búsqueda.

9.7. Interpretado frente a nativo

Misma instancia y configuración con `./horarios.sh` y con `./horarios_nativo` (compilado con `gplc`). El tiempo de CPU compara solo la búsqueda y el de pared, el costo total.

Instancia	Configuración	CPU (ms)		Pared (ms)	
		gprolog	gplc	gprolog	gplc
medio40	bt --heuristica=mrv	0.12 s	0.03 s	0.14 s	0.04 s
grande80	bt --heuristica=mrv	0.27 s	0.15 s	0.28 s	0.16 s

10. Análisis de la aceleración y de la eficiencia de la poda

La aceleración S medida es, en general, menor de lo que sugiere la eficiencia de poda E : una poda que evita el 99 por ciento de los nodos no da $100\times$, porque cada nodo de las versiones podadas cuesta más que uno de la versión ingenua.

Costo de evaluar las heurísticas en cada nodo. MRV recalcula `length/2` de los dominios de todas las sesiones pendientes en cada nodo. LCV cuenta, para *cada* valor candidato, cuántos valores elimina en *todas* las sesiones pendientes (`descartados/5`), con un costo por nodo del orden de $|\text{dom}| \cdot \sum_t |\text{dom}_t|$. Las heurísticas estáticas (`grado`, `demanda`) se calculan una vez, pero no reaccionan

a lo ya asignado. En `medio40`, FC + LCV tarda 1637ms frente a 27ms de FC + MRV, con los mismos 40 nodos; en `grande80`, 17162ms frente a 156ms (80 nodos en ambos). Son unas 60 y 110 veces más, porque la tasa de exploración cae de 1481 y 513 nodos por segundo con MRV a 24 y 5 con LCV.

Orden de las cláusulas y del dominio. `elegir/2` recorre el dominio con `member/2` en el orden en que `dominio/2` lo construyó (aula, día, franja), y el motor prueba los valores en ese orden fijo. Si los primeros valores de cada dominio son malos, la búsqueda retrocede mucho aunque haya heurísticas; por eso `original`, `grado` y `demanda` dan resultados distintos con la misma poda.

Costo de la propagación en CLP(FD). El modelo postea restricciones para *cada par* de sesiones (`fd_pares/2` es cuadrático), cada una con una disyunción o implicación reificada, además de los `fd_all_different/1` redundantes. Cada decisión de etiquetado dispara propagación sobre muchas de ellas, y cada retroceso del etiquetado binario agrega una restricción $X \neq W$.

Copia y recorrido de listas. `filtrar/5` reconstruye la lista de dominios pendientes tras cada asignación, y la verificación anticipada recorre con `member/2` todas las sesiones ya asignadas para cada valor candidato. Ambos costos son lineales por nodo y se pagan aunque la poda sea efectiva. En CLP(FD), `reconstruir/3` recorre las listas con `enesimo/3` cada vez que el branch and bound registra una hoja.

Uso de assert y retract. La búsqueda no los usa: los contadores son variables globales, mucho más baratas, que no se deshacen al retroceder. `assertz/1` y `retractall/1` se usan solo en la preparación (`preparar/0`, `preparar_indices/0`, `equiv_g/4`) y en el diagnóstico de culpables.

Lectura, análisis y validación del archivo. La lectura carácter a carácter y las pasadas de validación son un costo fijo por ejecución; la detección de duplicados compara todos los pares de registros (cuadrática). Es despreciable en instancias de decenas de registros, pero no cuando la búsqueda dura milisegundos, como en `mini.dat`.

Estructura de la instancia. En las ejecuciones con `bt` y MRV el número de nodos coincidió con el de sesiones (40 en `medio40`, 80 en `grande80`): no hubo retrocesos, y la aceleración viene de no equivocarse nunca, no de podar ramas. Los CSV lo confirman: FC + MRV exploró 10, 20, 40 y 80 nodos con 0 retrocesos en las cuatro instancias. (Con simetría activa sí hubo algunos retrocesos, por ejemplo 31 en `grande80`.) En instancias holgadas casi cualquier heurística razonable resuelve y las diferencias se reducen; en instancias muy restringidas hay poca poda posible y los nodos se agotan igual.

Interpretado frente a nativo. `gprolog` compila a bytecode los archivos fuente en cada ejecución y el código resultante se interpreta; `gplc` paga ese costo una sola vez y ejecuta código nativo. Por eso el tiempo de pared del nativo es menor incluso en búsquedas cortas, y el de CPU en las largas. Los tiempos reales medidos fueron los siguientes:

Instancia	Configuración	gprolog	gplc
<code>medio40</code>	<code>bt --heuristica=mrv</code>	0.14 s	0.04 s
<code>grande80</code>	<code>bt --heuristica=mrv</code>	0.28 s	0.16 s

Comparabilidad de E . E puede salir negativa si una configuración explora más nodos que la referencia, y si esta alcanza el límite es solo una cota inferior. Entre backtracking y CLP(FD) un nodo no es lo mismo, y con `fd_labeling/2` no hay conteo de nodos, así que E solo es comparable dentro de la misma familia.

11. Limitaciones del sistema

- **Un solo profesor por grupo.** `profesor_de/3` usa el primer profesor declarado en `IMPARTE`.
- **Huecos implícitos solo reportados.** No se optimizan; el costo del branch and bound solo incluye `prefiere` y `compacta`.
- **Branch and bound solo en CLP(FD).**
- **`fd_labeling/2` sin límite ni conteo de nodos.** En una instancia difícil puede no terminar.
- **LCV solo con etiquetado propio.**
- **Diagnóstico por restricción individual.** La culpable se busca quitando una a la vez, sin calcular un subconjunto mínimo, y con un límite de 200 000 nodos, por lo que una instancia resoluble pero costosa podría pasar inadvertida.
- **Disponibilidad más estricta que el enunciado.** Se exige en todas las franjas de la sesión, no solo en la de inicio.
- **Mensajes de validación genéricos en algunos casos.** Un rango de disponibilidad inválido se reporta como `profesor_invalido` sin precisar el campo.
- **Un aula por sesión, franjas de duración fija y días consecutivos.**
- **Comparabilidad de las métricas.** Los nodos de backtracking y de CLP(FD) no son equivalentes, y las medidas de milisegundos dependen de la resolución del reloj.